Module Responsibilities

Module	File	Responsibilities
Registration	`registration.py`	Registers crew with `name/role`; prevents duplicates; validates role names.
Crew Management	`crew_management.py`	Assigns/changes roles; stores skill levels; checks availability.
Inventory	`inventory.py`	Tracks cars, parts, tools, cash; tracks damage; manages part consumption.
Race Management	`race_management.py`	Creates races; validates drivers and race-ready cars; blocks on high heat.
Results	`results.py`	Records outcomes; updates ranking points; updates cash with prize; marks damage.
Mission Planning	`mission_planning.py`	Creates missions; validates required roles; blocks damaged-car missions; awards cash.
Garage	`garage.py`	Repairs damaged cars; requires mechanic role and `repair_kit` stock.
Heat Management	`heat_management.py`	Tracks heat accumulation; provides race-blocking state based on threshold.

Business Rule Integration

The following business rules from the specification are implemented and tested:

- A member must be **registered** before role assignment.
- Only members with role **driver** can enter races.
- Mission start **validates required roles**.
- If the linked mission car is damaged, **mechanic availability is required**.
- Race results **update inventory cash balance**.
- Mission **cannot start** if required roles are unavailable.

Call Graph & Control Flow Graph

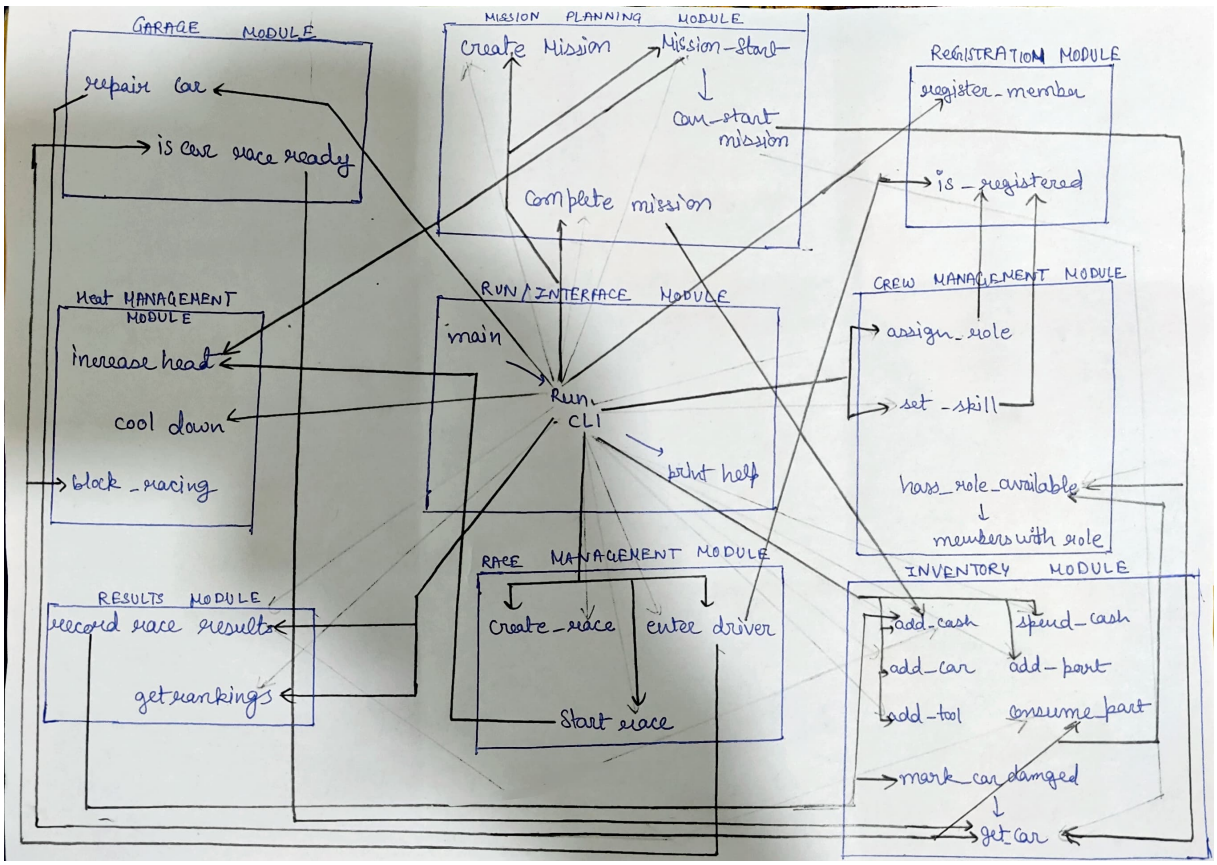

Figure 1: Call Graph showing inter-module function dependencies

Integration Test Case Matrix

ID	Scenario	Modules Involved	Expected Result	Result	Notes
IT-01	Register driver then enter race	Registration, Race Mgmt, Inventory	Entry accepted	**PASS**	None
IT-02	Enter race with unregistered driver	Race Mgmt, Registration	Reject with error	**PASS**	None
IT-03	Enter race with non-driver role	Registration, Crew Mgmt, Race Mgmt	Reject with error	**PASS**	None
IT-04	Assign role before registration	Crew Mgmt, Registration	Reject with error	**PASS**	Prevented invalid state
IT-05	Complete race updates cash & rankings	Race Mgmt, Results, Inventory	Cash + prize; rankings updated	**PASS**	None
IT-06	Record result before race start	Race Mgmt, Results	Reject with error	**PASS**	Enforced `running` state

ID	Scenario	Modules Involved	Expected Result	Result	Notes
IT-07	Damaged-car mission without mechanic	Mission Planning, Inventory, Crew Mgmt	Mission blocked	PASS	None
IT-08	Mission with missing required role	Mission Planning, Crew Mgmt	Mission blocked	PASS	None
IT-09	Mission with all required roles	Mission Planning, Crew Mgmt, Inventory	Starts, completes, reward added	PASS	None
IT-10	Repair with mechanic and parts	Garage, Crew Mgmt, Inventory	Damage reduced, part consumed	PASS	None
IT-11	Attempt repair without mechanic	Garage, Crew Mgmt	Repair blocked	PASS	None
IT-12	High heat blocks race entry	Heat Mgmt, Race Mgmt	Entry blocked	PASS	None
IT-13	End-to-end: race → damage → repair → mission	Race Mgmt, Results, Garage, Mission Planning, Inventory	All state transitions valid	PASS	None

Why These Integration Tests Are Needed

- **IT-01**: Ensures modules can perform the normal happy path across registration and race entry.
- **IT-02**: Checks data dependency — the race module must not accept unknown people.
- **IT-03**: Checks business rule enforcement: only drivers can race.
- **IT-04**: Verifies that role management depends on prior registration.
- **IT-05**: Checks cross-module data flow: race outcome must affect money and rankings.
- **IT-06**: Checks lifecycle correctness so results cannot be recorded for a race that never started.
- **IT-07**: Checks mission safety logic when required roles are missing due to a damaged linked car.
- **IT-08**: Checks mission blocking when a directly required role is unavailable.
- **IT-09**: Checks successful mission data flow and reward update.
- **IT-10**: Verifies repair dependencies (mechanic + part inventory) and car-state updates.

- **IT-11**: Verifies negative branch for unavailable mechanic.
- **IT-12**: Verifies global risk state (heat) correctly blocks race operations.
- **IT-13**: Verifies a realistic multi-module sequence — not just isolated function calls.

Test Results

All Integration Tests Passed

Total tests: **13**
Passed: **13**
Failed: **0**

All required module interactions behaved as expected.

Errors or Logical Issues Detected

No Runtime Integration Failures

No runtime integration failures were observed in the final test run. All 13 integration test cases passed on the first verified execution.